



UNIVERSIDAD
AUTÓNOMA
METROPOLITANA
Unidad Cuajimalpa



DCCD



PROYECTO FINAL SISTEMAS DISTRIBUIDOS

Mini Plataforma de Video Streaming P2P con Microservicios



MANUEL ALEJANDRO COLINDRES BARRIOS
PROFESOR: GUILLERMO MONROY RODRIGUEZ
TRIMESTRE: 25 P
MATRICULA: 2233030327

INTRODUCCIÓN

Este proyecto esta dedicado a construir una plataforma de streaming en un entorno peer-to-peer en donde 2 nodos van a compartir fragmentos de video entre si mediante APIs REST y sockets TCP, de esta forma no necesitamos de servicio externos como lo son Redis o MQTT.

Para desarrollarlo utilicé Python como lenguaje de programación principal y para los contenedores use Docker, asi simule dos nodos independientes que van a intercambiar fragmentos de video de manera autónoma pero coordinados.

Objetivo General

Construir un sistema distribuido que permita el intercambio de fragmentos de video entre nodos mediante peer-to-peer, usando microservicios para el manejo de los archivos, notificaciones simuladas con Pub/Sub y sincronización manual entre nodos, todo esto mediante Docker.

Objetivos Específicos

- Dividir un video en fragmentos manejables.
- Implementar una API REST en cada nodo para listar los fragmentos disponibles.
- Diseñar una arquitectura P2P para que los nodos intercambien los videos faltantes
- Simular un sistema Pub/Sub para notificar la disponibilidad de nuevos fragmentos.
- Usar los contenedores en Docker con cada nodo para simular su ejecución independiente.
- Registrar logs e interacciones a través de la terminal.

Tecnologías Utilizadas

- Python 3.11
- Flask
- Sockets TCP
- Docker, Docker Compose

- **Gestor de dependencias:** requirements.txt
- Swagger

Arquitectura del Sistema

El sistema está compuesto por dos nodos llamados node1 y node2, cada uno tiene su propia API REST, servidor de sockets TCP y cliente. Cada nodo contiene una carpeta con los fragmentos de video.

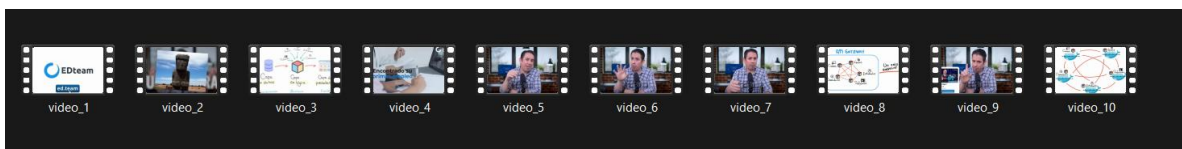
- Cada nodo expone un endpoint para listar los fragmentos disponibles.
- Cuando un nodo recibe un nuevo fragmento, lo anuncia a los suscriptores por TCP.
- Un script sincroniza los fragmentos faltantes entre nodos.

La comunicación puede ser iniciada manualmente desde consola o mediante comandos HTTP.

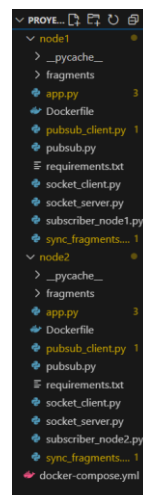
Proceso de Construcción

1. División de Video:

- Se dividió un video en 10 partes.



2. Diseño de nuestro proyecto



1. app.py (API Flask)

Provee endpoints REST para:

- Listar fragmentos (GET /fragments)
 - Descargar fragmentos (GET /fragments/{filename})
 - Subir nuevos fragmentos (POST /upload)
 - Notificaciones (POST /notify)
- Swagger
Documentación interactiva en <http://localhost:5000/apidocs>

```
app.py > force_sync
from flask import Flask, jsonify, request, send_from_directory
from flask_swagger import Swagger
import os
import threading
from sync_fragments import sincronizar_fragmentos

app = Flask(__name__)
app.config['PEER_URL'] = "http://node2:5000" # conexión automática a Nodo 2

# Swagger solo para subir archivos
swagger = Swagger(app, template={
    "info": {
        "title": "Nodo 1 - Carga de Fragmentos",
        "description": "Suba fragmentos para compartir",
        "version": "1.0"
    }
})

FRAGMENTS_DIR = 'fragments'

@app.route('/upload', methods=['POST'])
def upload():
    """
    Sube un fragmento y fuerza sincronización inmediata
    """
    tags:
    - Fragmentos
    consumes:
    - multipart/form-data
    parameters:
    - name: file
      in: formData
      type: file
      required: true
```

2. sync_fragments.py

Gestiona la sincronización automática mediante:

- Descarga de fragmentos faltantes vía HTTP o sockets
- Integración con pubsub para notificaciones

```

sync_fragments.py > ...
import os
import time
import requests

def sincronizar_fragmentos(app):
    while True:
        try:
            local_frags = set(f for f in os.listdir('fragments') if f.startswith('video'))
            peer = "node1" if os.environ.get('NODE_NAME') == 'node2' else "node2"

            print(f"\n🔄 Sincronizando con {peer}...")
            try:
                # 1. Obtener lista de fragmentos del peer
                response = requests.get(f"http://{peer}:5000/fragments")
                peer_frags = set(response.json().get('fragments', []))

                # 2. Descargar los faltantes
                for frag in peer_frags - local_frags:
                    print(f"📄 Descargando {frag}...")
                    r = requests.get(f"http://{peer}:5000/fragments/{frag}")
                    if r.status_code == 200:
                        with open(f'fragments/{frag}', 'wb') as f:
                            f.write(r.content)
                        print(f"✅ {frag} descargado!")
                    else:
                        print(f"❌ Error al descargar {frag}")

            except Exception as e:
                print(f"⚠️ Error de conexión: {str(e)}")

            time.sleep(15) # Sincroniza cada 15 segundos

        except Exception as e:
            print(f"💥 Error crítico: {str(e)}")

```

3. socket_server.py

Escucha conexiones TCP en un puerto dedicado (5001/5002) para:

- Recibir solicitudes de fragmentos
- Enviar archivos directamente (P2P)

```

socket_server.py > 📁 iniciar_servidor_socket
import socket
import time

def iniciar_servidor_socket(puerto):
    while True:
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.bind(('0.0.0.0', puerto))
                s.listen()
                print(f"Servidor socket P2P iniciado en puerto {puerto}")
                while True:
                    conn, addr = s.accept()
                    # ... manejo de conexiones ...
        except OSError as e:
            print(f"Error en servidor socket: {e}. Reintentando en 5 segundos...")
            time.sleep(5)

```

4. socket_client.py

Cliente para comunicación directa entre nodos:

```

socket_client.py > solicitar_fragmento_p2p
import socket
import json

def descargar_fragmento(host, puerto, fragmento, destino):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.settimeout(5)
            s.connect((host, puerto))

            solicitud = json.dumps({
                'accion': 'obtener_fragmento',
                'fragmento': fragmento
            })
            s.sendall(solicitud.encode())

            datos = b''
            while True:
                parte = s.recv(4096)
                if not parte:
                    break
                datos += parte

            if datos != b'Fragmento_no_encontrado':
                with open(destino, 'wb') as f:
                    f.write(datos)
                return True
            return False
    except Exception:
        return False

def solicitar_fragmento_p2p(fragmento):
    return descargar_fragmento('node2', 5001, fragmento, f'fragments/{fragmento}')

```

5. pubsub.py

Sistema de notificaciones interno

```

pubsub.py > PubSub > clear_subscriptions
class PubSub:
    def __init__(self):
        self.subscribers = dict()

    def publish(self, topic, message):
        """Notify all subscribers of a topic"""
        if topic in self.subscribers:
            for callback in self.subscribers[topic]:
                try:
                    callback(message)
                except Exception as e:
                    print(f"Error in subscriber callback: {e}")

    def subscribe(self, topic, callback):
        """Register a callback for a topic"""
        if topic not in self.subscribers:
            self.subscribers[topic] = []
        self.subscribers[topic].append(callback)

    def unsubscribe(self, topic, callback):
        """Remove a specific subscription"""
        if topic in self.subscribers:
            self.subscribers[topic] = [cb for cb in self.subscribers[topic] if cb != callback]

    def clear_subscriptions(self, topic):
        """Clear all subscriptions for a topic"""
        if topic in self.subscribers:
            del self.subscribers[topic]

```

6. docker-compose.yml

- Configuración:
 - Red aislada p2p-network
 - Mapeo de puertos:
 - Nodo1: 5000 (API), 5001 (P2P)

- Nodo2: 5002 (API), 5003 (P2P)
 - Volúmenes para persistencia de fragmentos

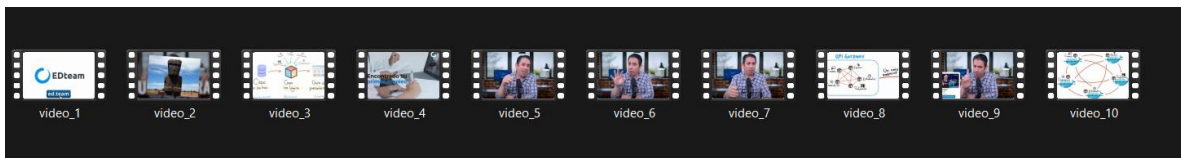
```

docker-compose.yml
1  version: '3.8'
2
3  services:
4    > Run Service
5    node1:
6      build: ./node1
7      ports:
8        - "5000:5000"
9      volumes:
10       - ./node1/fragments:/app/fragments
11      environment:
12        - NODE_NAME=node1
13
14    > Run Service
15    node2:
16      build: ./node2
17      ports:
18        - "5002:5000"
19      volumes:
20       - ./node2/fragments:/app/fragments
21      environment:
22        - NODE_NAME=node2
23
24  networks:
25    p2p-network:
26      driver: bridge

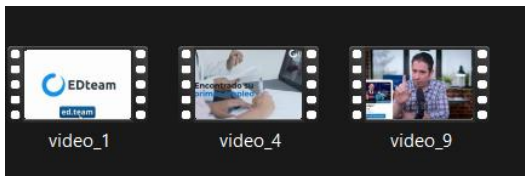
```

Pruebas del Sistema

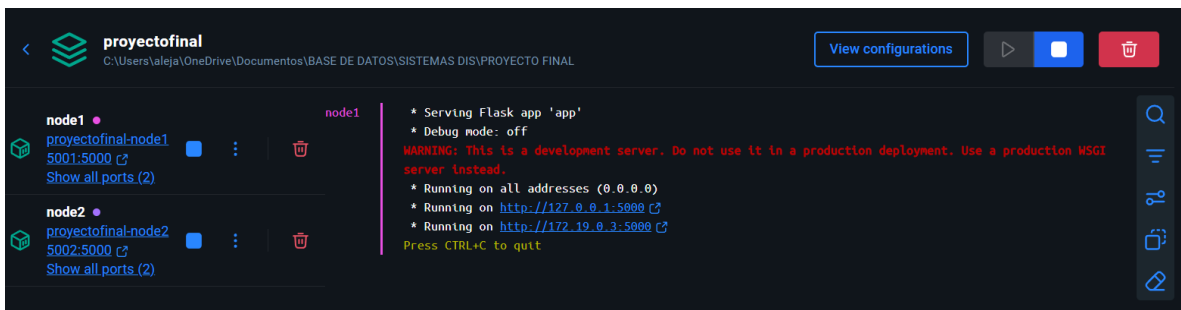
El nodo 1 tiene los 10 fragmentos



El nodo 2 tiene solo 3 fragmentos



Construir los contenedores.



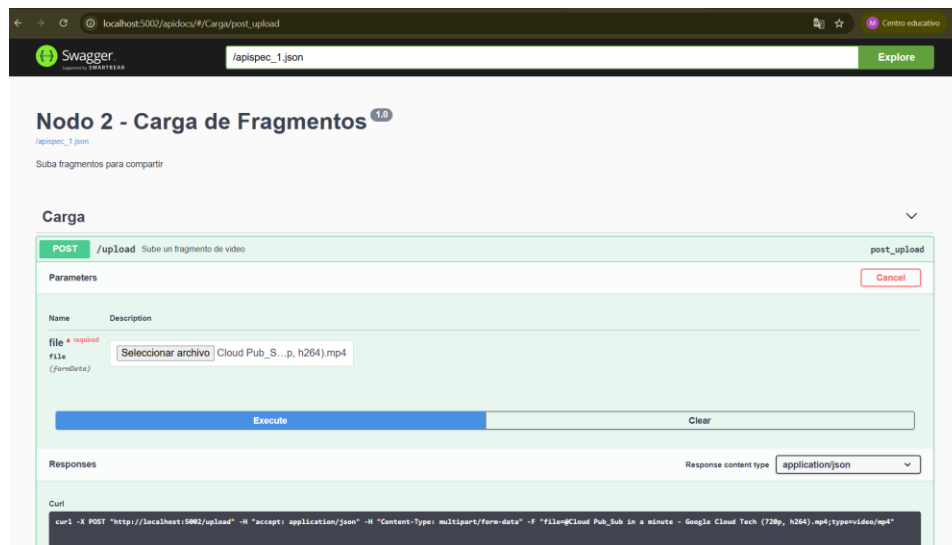
Al levantar los contenedores se activa la función de sync_fragments que es la que compara y guarda los fragmentos que no estén.

```
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:26] "GET /fragments/video_2.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:27] "GET /fragments/video_3.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:29] "GET /fragments/video_5.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:30] "GET /fragments/video_6.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:31] "GET /fragments/video_7.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [08/Aug/2025 23:07:33] "GET /fragments/video_8.mp4 HTTP/1.1" 200 -
```

Si se llegan a borrar o ausbir nuevos, se continua el proceso de obtener fragmento

```
node1-1 | 172.21.0.3 - - [09/Aug/2025 00:06:44] "GET /fragments HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [09/Aug/2025 00:06:44] "GET /fragments/video_6.mp4 HTTP/1.1" 200 -
node1-1 | 172.21.0.3 - - [09/Aug/2025 00:06:45] "GET /fragments/video_5.mp4 HTTP/1.1" 200 -
node2-1 | 172.21.0.2 - - [09/Aug/2025 00:06:48] "GET /fragments HTTP/1.1" 200 -
```

Mediante Swagger se pueden subir nuevos videos



Conclusiones

Este proyecto demuestra que es posible simular una plataforma distribuida para intercambiar videos usando tecnologías accesibles como Python, Flask y sockets TCP. Asi se logra tener una arquitectura funcional basada en microservicios y ciertos principios de P2P.

El uso de Docker permite simular entornos aislados, facilitando el despliegue y las pruebas. Este sistema podría escalarse fácilmente para incluir más nodos, autenticación, verificación de integridad de fragmentos, e interfaces gráficas más amigables.